LAPORAN PRAKTIKUM

Aplikasi Berbasis Platform

Tugas pertemuan 8 git

branch

Disusun oleh:

ZEFANYA BRANA TERTIUS TARIGAN

2311102028

S1 IF-11-REG04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

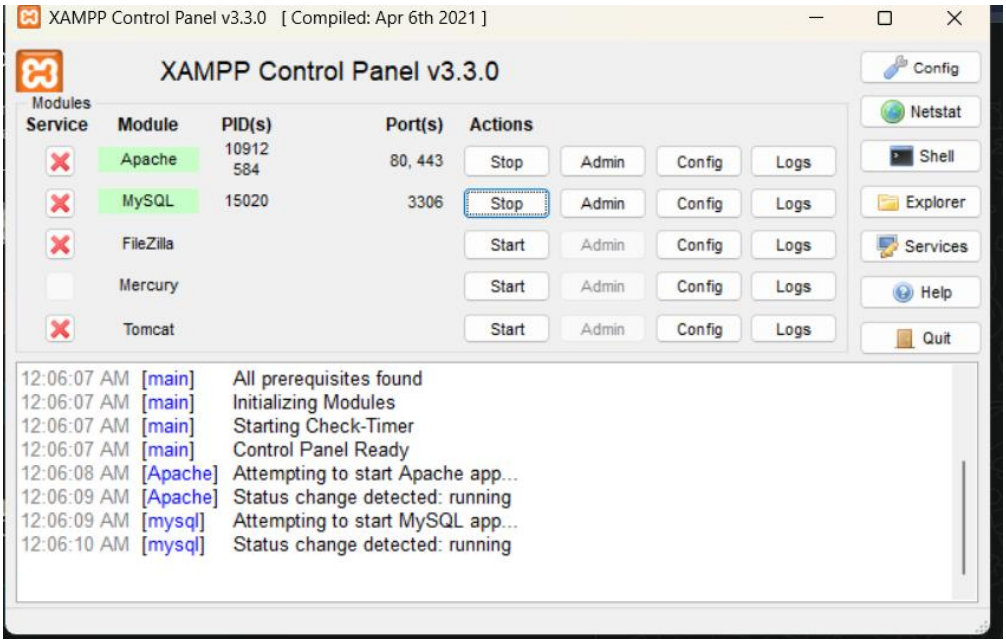
TELKOM UNIVERSITY

2026/2027

GUIDED

Modul 12

1.Nyalakan Apache dan Mysql pada XAMPP



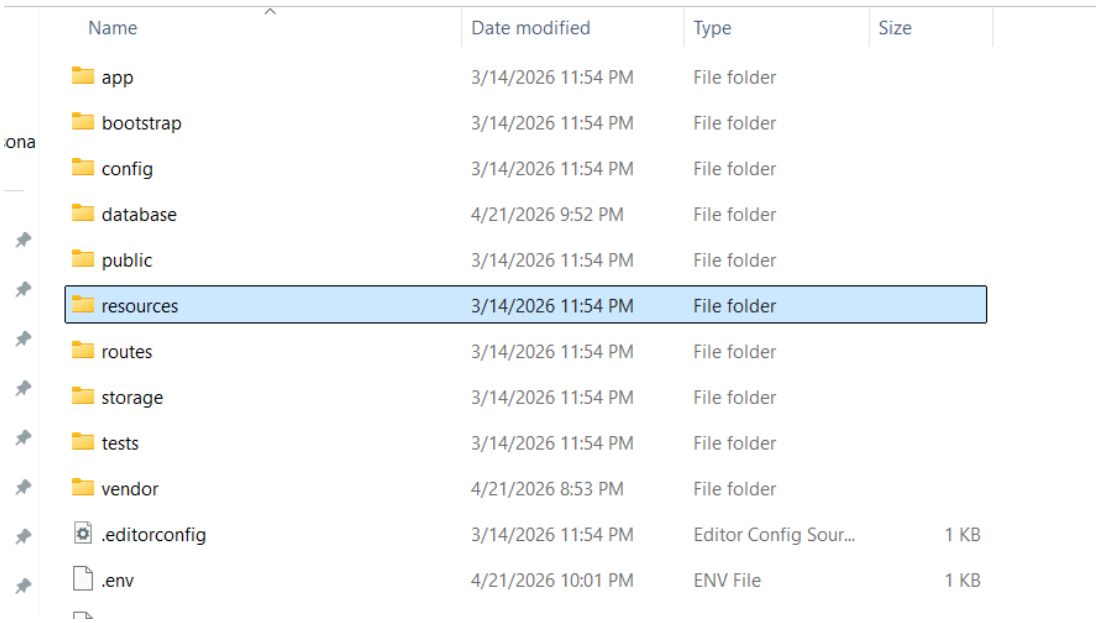
2. **Mengatur koneksi database**

Konfigurasi database dilakukan pada file .env sesuai petunjuk modul, yaitu mengatur DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, dan DB_PASSWORD. Database yang digunakan adalah **ecommerce**.

3. **Membuat database ecommerce**

Setelah konfigurasi .env, dibuat database baru bernama **ecommerce** pada phpMyAdmin agar Laravel dapat terhubung ke database yang sesuai.

4.Sudah membuat folder seperti pada gambar di bawah ini.



Name	Date modified	Type	Size
app	3/14/2026 11:54 PM	File folder	
bootstrap	3/14/2026 11:54 PM	File folder	
config	3/14/2026 11:54 PM	File folder	
database	4/21/2026 9:52 PM	File folder	
public	3/14/2026 11:54 PM	File folder	
resources	3/14/2026 11:54 PM	File folder	
routes	3/14/2026 11:54 PM	File folder	
storage	3/14/2026 11:54 PM	File folder	
tests	3/14/2026 11:54 PM	File folder	
vendor	4/21/2026 8:53 PM	File folder	
.editorconfig	3/14/2026 11:54 PM	Editor Config Sour...	1 KB
.env	4/21/2026 10:01 PM	ENV File	1 KB

4. Edit file migration create_products_table

```
<?php


use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;


return new class extends Migration
{
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('name');
```

```
            $table->integer('price');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
```

Jalankan migration

Di cmd:

```
php artisan migrate
```

Buat model Product

Di CMD:

```
php artisan make:model Product
```

Buka file:

(app/Models/Product.php)

Ganti isi:

```
<?php


namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Model;


class Product extends Model
{
    protected $fillable = ['name', 'price'];
}
```

Buat controller resource

Di CMD:

```
     php artisan make:controller ProductController --resource
```

Buka file:

(app/Http/Controllers/ProductController.php)

Ganti isi dengan:

```
<?php


namespace App\Http\Controllers;


use App\Models\Product;
use Illuminate\Http\Request;


class ProductController extends Controller
{
    public function index()
    {
        $products = Product::all();
```

```
    return view('products.index', compact('products'));
}

public function create()
{
    return view('products.form', [
        'title' => 'Tambah',
        'product' => new Product(),
        'route' => route('products.store'),
        'method' => 'POST',
    ]);
}

public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);

    Product::create($validated);

    return redirect()->route('products.index')
        ->with('success', 'Produk berhasil ditambahkan');
}

public function show(Product $product)
```

```
{
    //
}

public function edit(Product $product)
{
    return view('products.form', [
        'title' => 'Edit',
        'product' => $product,
        'route' => route('products.update', $product->id),
        'method' => 'PUT',
    ]);
}

public function update(Request $request, Product $product)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);

    $product->update($validated);

    return redirect()->route('products.index')
        ->with('success', 'Produk berhasil diupdate');
}
```

```
    public function destroy(Product $product)
    {
        $product->delete();

        return redirect()->route('products.index')
            ->with('success', 'Produk berhasil dihapus');
    }
}
```

Tambahkan route resource

Buka file:

(routes/web.php)

Ganti isi:

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\ProductController;

Route::resource('products', ProductController::class);
```

Buat folder view

Buat folder:

(resources/views/products)

Isi:

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\ProductController;

Route::resource('products', ProductController::class);
```

Buat folder view

Buat folder:

(resources/views/products)

Di dalamnya buat 2 file:

- index.blade.php
- form.blade.php

Isi index.blade.php

Buka:

(resources/views/products/index.blade.php)

Isi:

```
@extends('template')

@section('title', 'Daftar Produk')

@section('content')
<div class="container mt-5">
    <h2>Daftar Produk</h2>
```

```
@if(session('success'))
    <div class="alert alert-success">{{ session('success') }}</div>
@endif

<a href="{{ route('products.create') }}" class="btn btn-primary mb-3">Tambah
Produk</a>

<table class="table table-bordered">
    <thead>
        <tr>
            <th>Nama</th>
            <th>Harga</th>
            <th width="200">Aksi</th>
        </tr>
    </thead>
    <tbody>
        @forelse($products as $product)
            <tr>
                <td>{{ $product->name }}</td>
                <td>{{ $product->price }}</td>
                <td>
                    <a href="{{ route('products.edit', $product->id) }}" class="btn btn-
warning btn-sm">Edit</a>

                    <form action="{{ route('products.destroy', $product->id) }}"
method="POST" style="display:inline;">
                        @csrf
```

```
                    @method('DELETE')
                    <button type="submit" class="btn btn-danger btn-sm"
onclick="return confirm('Yakin hapus data?')">Hapus</button>
                </form>
            </td>
        </tr>
    @empty
        <tr>
            <td colspan="3">Belum ada data produk.</td>
        </tr>
    @endforelse
  </tbody>
 </table>
</div>
@endsection
```

Isi form.blade.php

Buka:

(resources/views/products/form.blade.php)

Isi:

```
@extends('template')


@section('title', $title . ' Produk')


@section('content')
<div class="container mt-5">
```

```
<h2>{{ $title }} Produk</h2>


<form action="{{ $route }}" method="POST">
    @csrf
    @if($method == 'PUT')
        @method('PUT')
    @endif


    <div class="mb-3">
        <label class="form-label">Nama Produk</label>
        <input type="text" name="name" class="form-control @error('name') is-invalid
@enderror" value="{{ old('name', $product->name) }}">
        @error('name')
            <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>


    <div class="mb-3">
        <label class="form-label">Harga</label>
        <input type="number" name="price" class="form-control @error('price') is-
invalid @enderror" value="{{ old('price', $product->price) }}">
        @error('price')
            <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>


    <button type="submit" class="btn btn-success">Simpan</button>
```

```
        <a href="{{ route('products.index') }}" class="btn btn-secondary">Kembali</a>
    </form>
</div>
@endsection
```

Buat file template

Buka atau buat file:

(resources/views/template.blade.php)

Isi:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title')</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body style="width:95%">
    @yield('content')


    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></sc
ript>
</body>
</html>
```

Bersihkan cache Laravel

Di CMD:

```
php artisan optimize:clear
```

Jalankan aplikasi

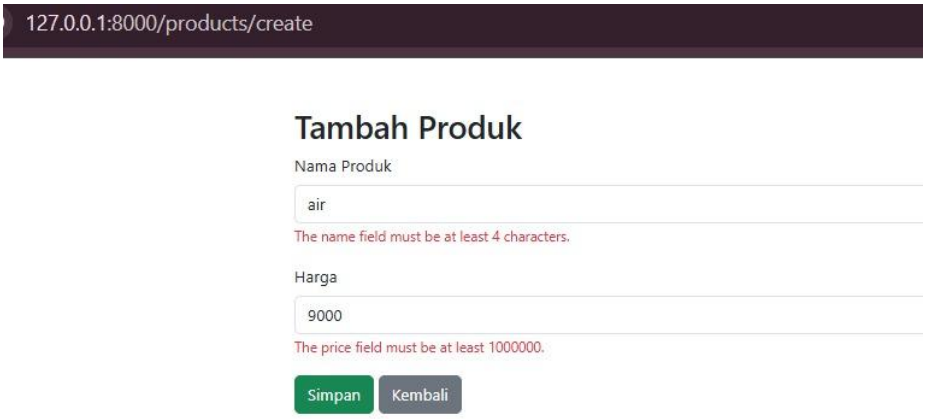
Di CMD:

```
php artisan serve
```

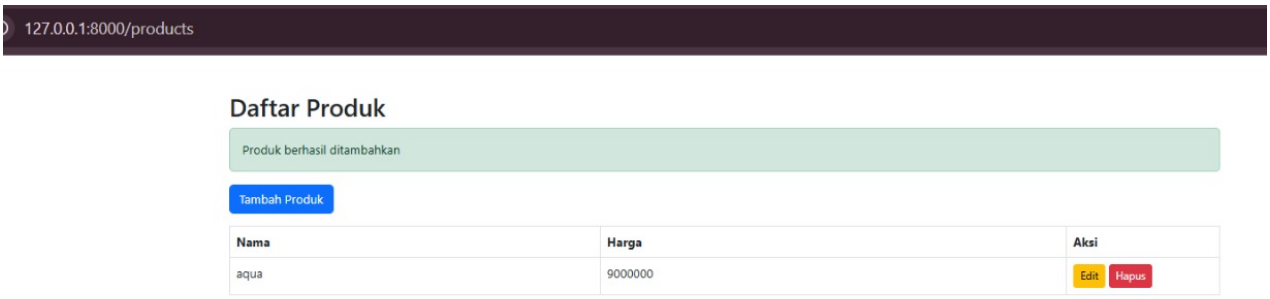
Lalu buka di browser:

(http://127.0.0.1:8000/products)

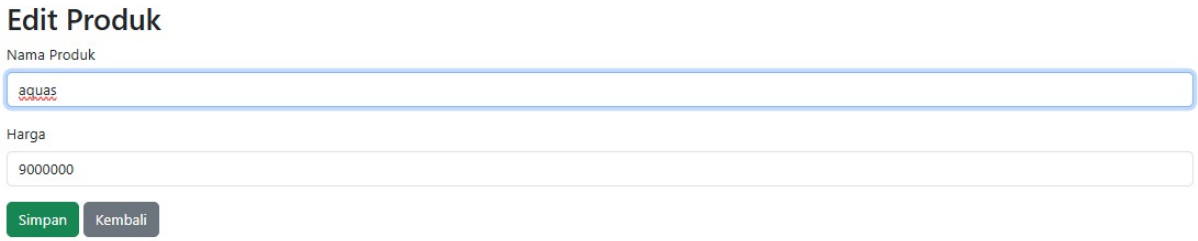
Output:



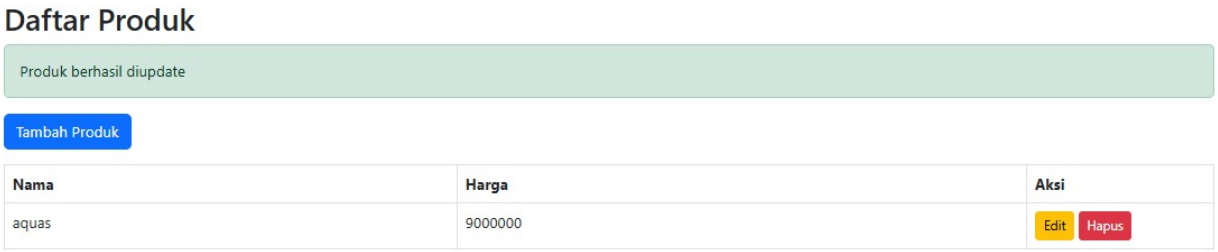
Tampilan untuk tes apakah bisa,ternyata harus ada kriteria yang dipenuhi.



Ini tampilan setelah berhasil menambahkan nya.



Ini tampilan edit



Setelah di edit.

Ini untuk menghapus.

GUIDED

MODUL 13

Langkah-Langkah Pengerjaan

1. Menyiapkan SiteController

Langkah pertama adalah membuat controller untuk menangani proses login.

cmd:

```
php artisan make:controller SiteController
```

File yang di edit:

(app/Http/Controllers/SiteController.php)

Kode:

```
<?php


namespace App\Http\Controllers;


use Illuminate\Http\Request;

use Illuminate\Support\Facades\Auth;
```

```
class SiteController extends Controller

{

    public function auth(Request $req)

    {

        if (Auth::attempt([

            'email' => $req->em,

            'password' => $req->pwd

        ])) {

            return redirect('/products');

        }



        return redirect('/login')->with('msg', 'Email / password salah');

    }

}
```

Membuat halaman login

Selanjutnya dibuat halaman login sebagai tampilan awal autentikasi.

File yang ditambah

(resources/views/login.blade.php)

Kode:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Login</title>

    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

</head>

<body class="bg-light">

    <div class="container">

        <div class="row justify-content-center align-items-center min-vh-100">

            <div class="col-md-4">

                <div class="card">

                    <div class="card-body">

                        <h3 class="card-title text-center mb-4">Login</h3>
```

```
                @if(session('msg'))
                    <div class="alert alert-warning">{{ session('msg') }}</div>
                @endif


                <form method="POST" action="{{ route('auth') }}">
                    @csrf
                    <div class="mb-3">
                        <label for="em" class="form-label">Email</label>
                        <input type="email" name="em" id="em" class="form-control"
required>
                    </div>


                    <div class="mb-3">
                        <label for="pwd" class="form-label">Password</label>
                        <input type="password" name="pwd" id="pwd" class="form-
control" required>
                    </div>


                    <div class="d-grid">
                        <button type="submit" class="btn btn-primary">Login</button>
                    </div>
                </form>
```

```
                </div>

            </div>

        </div>

    </div>

  </div>

</body>

</html>
```

Mengatur routing login, logout, dan proteksi halaman product

Routing login dan logout diatur, lalu halaman product diproteksi dengan middleware auth. Pada modul, route login diberi alias karena dibutuhkan oleh middleware auth, Auth::logout() dipakai untuk logout, dan halaman product diproteksi dengan middleware.

File yang diedit

(routes/web.php)

kode:

```
<?php


use Illuminate\Support\Facades\Route;

use Illuminate\Support\Facades\Auth;

use App\Http\Controllers\ProductController;
```

```
use App\Http\Controllers\SiteController;


Route::get('/login', function () {
    if (Auth::check()) return redirect('/products');
    return view('login');
})->name('login');


Route::post('/auth', [SiteController::class, 'auth'])->name('auth');


Route::get('/logout', function () {
    Auth::logout();
    return redirect('/login');
});


Route::resource('products', ProductController::class)->middleware('auth');
```

Mengubah template agar menampilkan user yang login

Setelah Auth dipakai, modul menyarankan mengganti pengecekan session biasa menjadi directive Blade @auth, lalu menampilkan nama user melalui Auth::user()->name.

File yang diedit

(resources/views/template.blade.php)

Kode:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>@yield('title')</title>

    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

</head>

<body style="width:95%">



    @auth

        <div class="row justify-content-end" style="margin-top:2%">

            <div class="col-3">

                {{ Auth::user()->name }}
```

```
            <a href="/logout" class="btn btn-warning">Logout</a>

        </div>

    </div>

  @endauth



  <div class="row justify-content-center" style="margin-top:10%">

    @yield('content')

  </div>



  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></sc
ript>

</body>

</html>
```

Menambahkan user untuk uji login

Karena Auth::attempt() membutuhkan user pada tabel users dan password harus di-hash dengan Bcrypt, maka user uji ditambahkan lewat Tinker.

CMD

```
    php artisan tinker
```

perintah di Tinker

```
\App\Models\User::create([

    'name' => 'User Auth',

    'email' => 'auth@test.com',

    'password' => bcrypt('123456')

]);
```

Keluar dari Tinker:

```
exit
```

Membersihkan cache dan menjalankan aplikasi

Setelah perubahan route, view, dan controller selesai, cache Laravel dibersihkan lalu server dijalankan.

CMD

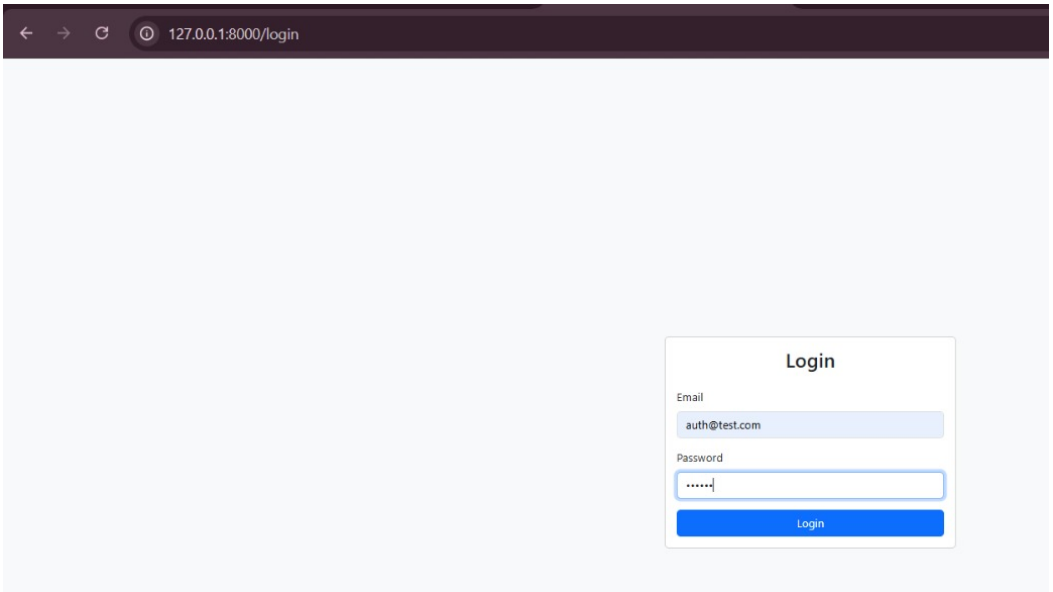
```
php artisan optimize:clear
```

```
php artisan serve
```

Kemudian aplikasi diuji pada:

(http://127.0.0.1:8000/login)

Output:



Membuat model dan migration Variant

Pada bagian 13.3, modul menjelaskan bahwa relasi yang dicoba adalah one-to-many antara Product dan Variant. Langkah awalnya adalah membuat model dan migration Variant.

CMD:

```
php artisan make:model Variant -m
```

Mengedit migration create_variants_table

Setelah file migration dibuat, struktur tabel variants didefinisikan dengan kolom:

- name
- description
- processor
- memory
- storage
- product_id
- timestamps

Kode:

```
<?php


use Illuminate\Database\Migrations\Migration;

use Illuminate\Database\Schema\Blueprint;

use Illuminate\Support\Facades\Schema;


return new class extends Migration

{
```

```
    public function up(): void
    {
        Schema::create('variants', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->text('description');
            $table->string('processor');
            $table->string('memory');
            $table->string('storage');
            $table->foreignId('product_id')->constrained();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('variants');
    }
};
```

Menjalankan migration variants

CMD

```
php artisan migrate
```

Mengedit model Variant

Modul menjelaskan bahwa tiap Variant hanya dimiliki oleh satu Product, sehingga dipakai relasi belongsTo(Product::class).

File yang diedit

(app/Models/Variant.php)

Kode:

```
<?php


namespace App\Models;


use Illuminate\Database\Eloquent\Model;


class Variant extends Model
{
    protected $fillable = [
        'name',
        'description',
```

```
        'processor',
        'memory',
        'storage',
        'product_id'
    ];

    public function product()
    {
        return $this->belongsTo(Product::class);
    }
}
```

Mengedit model Product

Karena satu Product dapat memiliki banyak Variant, maka model Product ditambahkan relasi hasMany(Variant::class).

File yang diedit

(app/Models/Product.php)

Kode:

```
<?php


namespace App\Models;


use Illuminate\Database\Eloquent\Model;


class Product extends Model
{
    protected $fillable = ['name', 'price'];


    public function variants()
    {
        return $this->hasMany(Variant::class);
    }
}
```

Menambahkan data Product dan Variant untuk pengujian

Karena foreign key product_id harus mengarah ke ID product yang benar-benar ada, data Product dibuat lebih dulu. Setelah itu variant ditambahkan melalui relasi $p->variants()->create(...).

CMD

```
php artisan tinker
```

Perintah pada Tinker

```
$p = \App\Models\Product::create([
    'name' => 'Produk Utama',
    'price' => 9000000
]);


$p->variants()->create([
    'name' => 'Variant 1',
    'description' => 'Laptop untuk kerja',
    'processor' => 'Intel i5',
    'memory' => '8GB',
    'storage' => '512GB SSD'
]);
```

```
$p->variants()->create([

    'name' => 'Variant 2',

    'description' => 'Laptop untuk desain',

    'processor' => 'Intel i7',

    'memory' => '16GB',

    'storage' => '1TB SSD'

]);
```

keluar:

```
exit
```

Mengubah tampilan index.blade.php agar menampilkan variant

Untuk membuktikan relasi berjalan, halaman daftar product diubah dengan menambahkan kolom **Variant**. Kolom ini menampilkan semua variant milik product dan nama product yang terkait dari masing-masing variant.

File yang diedit

(resources/views/products/index.blade.php)

kode:

```
@extends('template')



@section('title', 'Daftar Produk')
```

```
@section('content')

<div class="container mt-5">

    <h2>Daftar Produk</h2>



    @if(session('success'))

        <div class="alert alert-success">{{ session('success') }}</div>

    @endif



    <a href="{{ route('products.create') }}" class="btn btn-primary mb-3">Tambah
Produk</a>



    <table class="table table-bordered">

        <thead>

            <tr>

                <th>Nama</th>

                <th>Harga</th>

                <th>Variant</th>

                <th width="200">Aksi</th>

            </tr>

        </thead>

        <tbody>

            @forelse($products as $product)
```

```
<tr>
    <td>{{ $product->name }}</td>
    <td>{{ $product->price }}</td>
    <td>
        @if($product->variants->count())
            <ul class="mb-0">
                @foreach($product->variants as $var)
                    <li>
                        <strong>{{ $var->name }}</strong><br>
                        Desc: {{ $var->description }} <br>
                        Proc: {{ $var->processor }} <br>
                        RAM: {{ $var->memory }} <br>
                        Strg: {{ $var->storage }} <br>
                        Product: {{ $var->product->name }}
                    </li>
                    <br>
                @endforeach
            </ul>
        @else
            <span class="text-muted">Belum ada variant</span>
        @endif
```

```
                </td>
                <td>
                    <a href="{{ route('products.edit', $product->id) }}" class="btn btn-
warning btn-sm">Edit</a>


                    <form action="{{ route('products.destroy', $product->id) }}"
method="POST" style="display:inline;">
                        @csrf
                        @method('DELETE')
                        <button type="submit" class="btn btn-danger btn-sm"
onclick="return confirm('Yakin hapus data?')">Hapus</button>
                    </form>
                </td>
            </tr>
        @empty
            <tr>
                <td colspan="4">Belum ada data produk.</td>
            </tr>
        @endforelse
    </tbody>
  </table>
</div>
```

```
@endsection
```

Mengubah ProductController pada method index()

Agar relasi dimuat lebih aman, method index() pada ProductController diubah menjadi eager loading.

File yang diedit

(app/Http/Controllers/ProductController.php)

Kode:

```
public function index()

{

    $products = Product::with('variants.product')->get();

    return view('products.index', compact('products'));

}
```

Membersihkan cache dan menjalankan ulang aplikasi

CMD

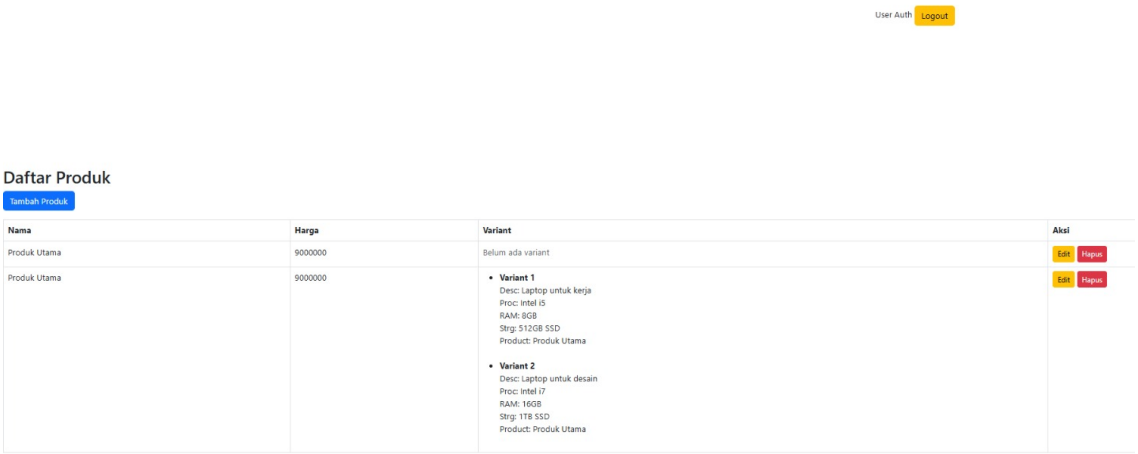
```
php artisan optimize:clear

php artisan serve
```

kemudian halaman di uji pada products:

(http://127.0.0.1:8000/products)

Output:



UNGUIDED

1. Apa itu git branch

= Dengan fitur git branch, user dapat membuat cabang kerja yang terpisah dari proyek utama, yang memungkinkan user mengerjakan fitur baru, memperbaiki bug, atau mencoba perubahan tanpa mengganggu cabang utama seperti main. Di dokumentasi Git, cabang dijelaskan sebagai model kerja yang sangat ringan, yang memungkinkan Anda membuat dan berpindah cabang dengan cepat.

- buatlah git branch dengan 2 akun berbeda dan hubungkan dengan project yang di buat di tugas 2 (bisa dengan antar teman kelas

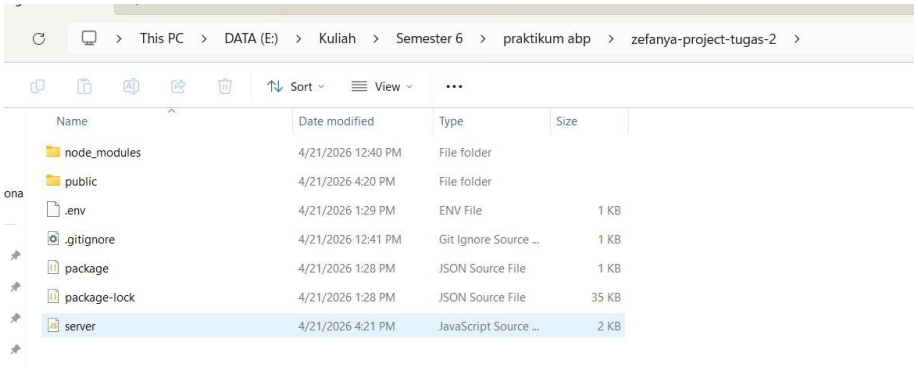
- kalian jelaskan apa saja fungsi nya dan apa keuntungan git branch

- buat juga output dan input apa saja yang dapat kalian lakukan mengunakan git branch

2. buatlah git branch dengan 2 akun berbeda dan hubungkan dengan project yang di buat di tugas 2 (bisa dengan antar teman kelas

=

Buat folder, isi nya seperti gambar ini



pada cmd:

1.Untuk membuat folder nya,ketik perintah ini.

(cd /d "E:\Kuliah\Semester 6\praktikum abp\zefanya-project-tugas-2")

2.Inisialisasi Git

```
git init

git branch -M main
```

3. Menambahkan semua file project ke staging area

```
git add .
```

4. Commit awal project

```
git commit -m "Membuat website dan koneksi PostgreSQL localhost"
```

5. Menambahkan remote repository GitHub

```
git remote add origin https://github.com/Zefanyatarigan/Modul-git-branch.git
```

6. Push branch main ke GitHub

```
git push -u origin main --force
```

Pengerjaan branch akun pertama

1.Membuat branch backend

```
git switch -c fitur-backend
```

2.Push branch backend ke GitHub

```
git push -u origin fitur-backend
```

3.Commit perubahan backend

```
git add .
git commit -m "Menambahkan endpoint status pada backend"
git push
```

4. Saat terjadi konflik merge

```
git switch fitur-backend
git fetch origin git
merge origin/main
```

5. Setelah konflik diperbaiki

```
git add server.js
git commit -m "Menyelesaikan conflict pada branch backend"
git push
```

6. Sinkronisasi branch utama setelah semua merge selesai

```
git switch main git
pull origin main
```

7. Commit final perapian project

```
git add server.js
git commit -m "Merapikan server.js setelah merge conflict" git push
```

Pengerjaan branch akun kedua

Pada tugas ini digunakan **2 akun GitHub pada 1 device**, yaitu:

- **Akun 1:** Zefanyatarigan
- **Akun 2:** Hawishh

Karena menggunakan 1 device, akun kedua mengerjakan branch frontend langsung melalui **GitHub Web**.

Langkah akun kedua

1. Akun pertama mengundang akun kedua sebagai collaborator.
2. Akun kedua menerima invitation.
3. Akun kedua membuat branch fitur-frontend dari branch main.
4. Akun kedua mengedit:
    - public/index.html
    - public/style.css
5. Akun kedua commit perubahan di branch fitur-frontend.
6. Akun kedua membuat Pull Request fitur-frontend ke main.
7. Pull Request di-merge.

Fungsi Git Branch

Fungsi Git branch pada project ini adalah:

1. Memisahkan pekerjaan backend dan frontend.
2. Menghindari perubahan langsung pada branch utama.
3. Memudahkan kolaborasi antara 2 akun berbeda.
4. Mempermudah proses penggabungan fitur setelah selesai. 5. Menjaga project utama tetap aman selama pengembangan.

Keuntungan Git Branch

Keuntungan penggunaan Git branch:

1. Pengerjaan lebih rapi dan terstruktur.
2. Kesalahan di satu branch tidak langsung merusak main.
3. Tiap anggota bisa fokus pada bagian masing-masing.
4. Mempermudah review sebelum digabung ke branch utama.
5. Memudahkan penelusuran perubahan berdasarkan fitur.

Input dan output pada Git Branch

1. Membuat branch

Input

```
git switch -c fitur-backend
```

Output

```
Switched to a new branch 'fitur-backend'
```

2.Melihat daftar branch

Input:

```
git branch
```

Output:

```
* fitur-backend
main
```

3. Push branch ke GitHub

Input

```
git push -u origin fitur-backend
```

Output

```
branch 'fitur-backend' set up to track 'origin/fitur-backend'
```

4. Melihat status project

Input

```
git status
```

Output

```
On branch main

Your branch is up to date with 'origin/main'.


nothing to commit, working tree clean
```

5. Sinkronisasi dengan branch utama

Input

```
git switch main git
pull origin main
```

Output

Output menampilkan proses pengambilan update dari repository GitHub ke branch lokal main.

Membuat website dan menambahkan database yang terhubung ke localhost

A. Struktur project

Project dibuat dengan komponen berikut:

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js + Express
- Database: PostgreSQL localhost

1. Membuat project Node.js

```
npm init -y
```

2. Menginstall package yang dibutuhkan

```
npm install express pg cors dotenv
```

Perintah membuat database PostgreSQL localhost

1.Membuat database

```
CREATE DATABASE db_tugas2;
```

2.Membuat tabel produk

```
CREATE TABLE produk (     id
SERIAL PRIMARY KEY,
    nama_produk VARCHAR(100) NOT NULL,
harga INTEGER NOT NULL,    stok INTEGER
NOT NULL
);
```

Konfigurasi file .env

Isi file .env:

```
DB_USER=postgres

DB_HOST=localhost

DB_NAME=db_tugas2

DB_PASSWORD=PASSWORD_POSTGRES_KAMU
DB_PORT=5432

PORT=3000
```

Isi file server.js

```
const express = require('express');
const { Pool } = require('pg');
const cors = require('cors');
require('dotenv').config();


const app = express();


app.use(cors());
app.use(express.json());
app.use(express.static('public'));


const pool = new Pool({   user:
process.env.DB_USER,   host:
process.env.DB_HOST,   database:
process.env.DB_NAME,   password:
process.env.DB_PASSWORD,   port:
process.env.DB_PORT,
});


app.get('/api/produk', async (req, res) => {
try {
   const result = await pool.query('SELECT * FROM produk ORDER BY id
ASC');     res.json(result.rows);   } catch (error) {
   res.status(500).json({ error: error.message });
}
```

```
});


app.post('/api/produk', async (req, res) => {
try {
    const { nama_produk, harga, stok } = req.body;
const result = await pool.query(
      'INSERT INTO produk (nama_produk, harga, stok) VALUES ($1, $2, $3) RETURNING
*',
      [nama_produk, harga, stok]
    );
    res.json({
      message: 'Data berhasil ditambahkan',
data: result.rows[0]
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
}
});


app.get('/api/status', (req, res) => {
  res.json({ message: 'Backend berjalan dengan baik' }); });


app.listen(process.env.PORT, () => {
  console.log(`Server berjalan di http://localhost:${process.env.PORT}`); });
```

Menjalankan project

```
node server.js
```

Setelah itu masuk ke website

ketik perintah

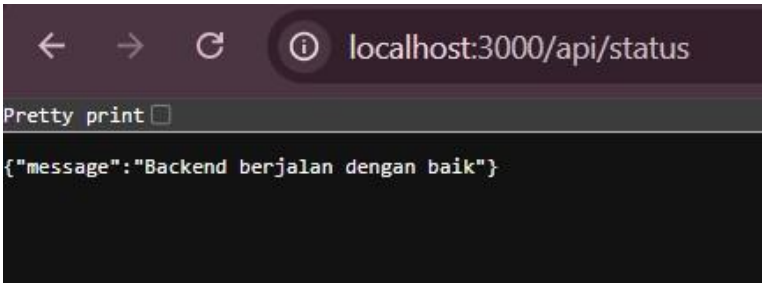
1. Website utama

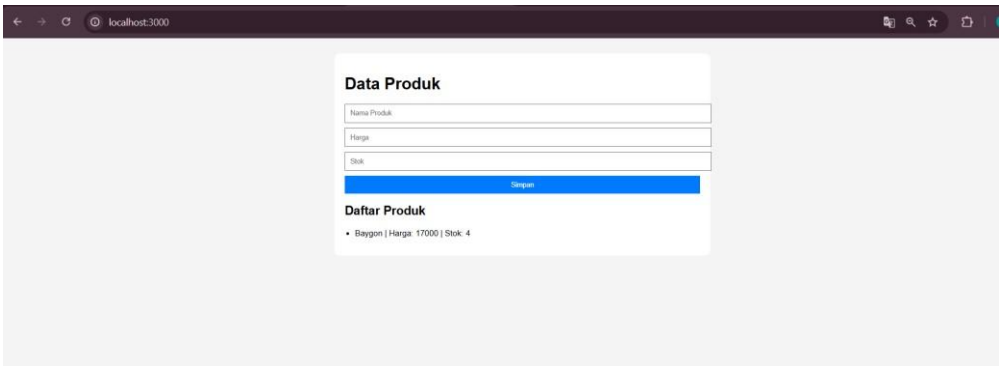
```
http://localhost:3000
```

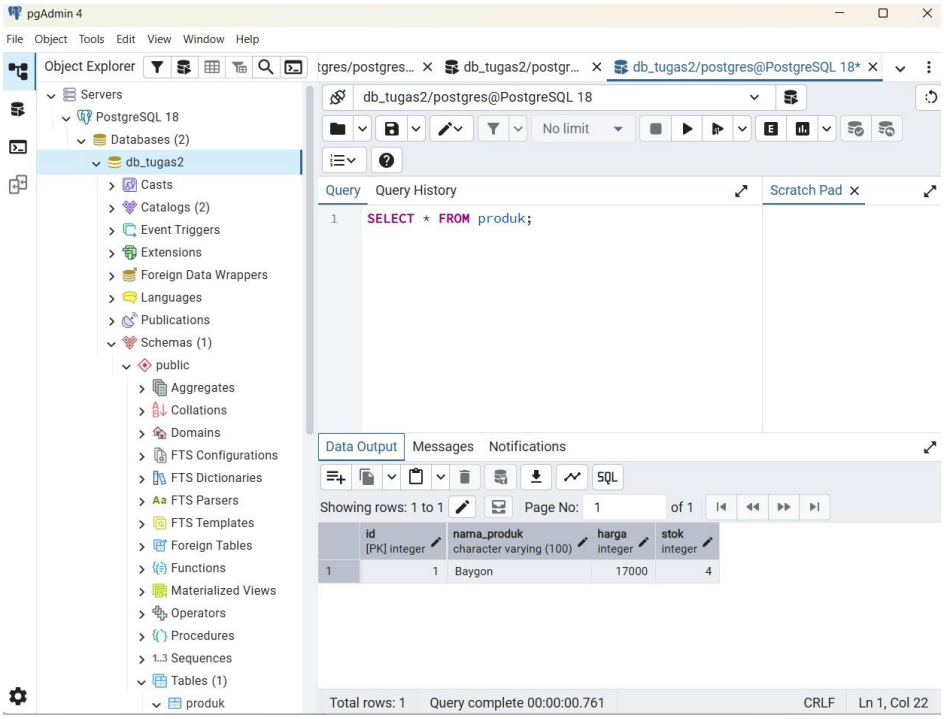
2. Endpoint backend

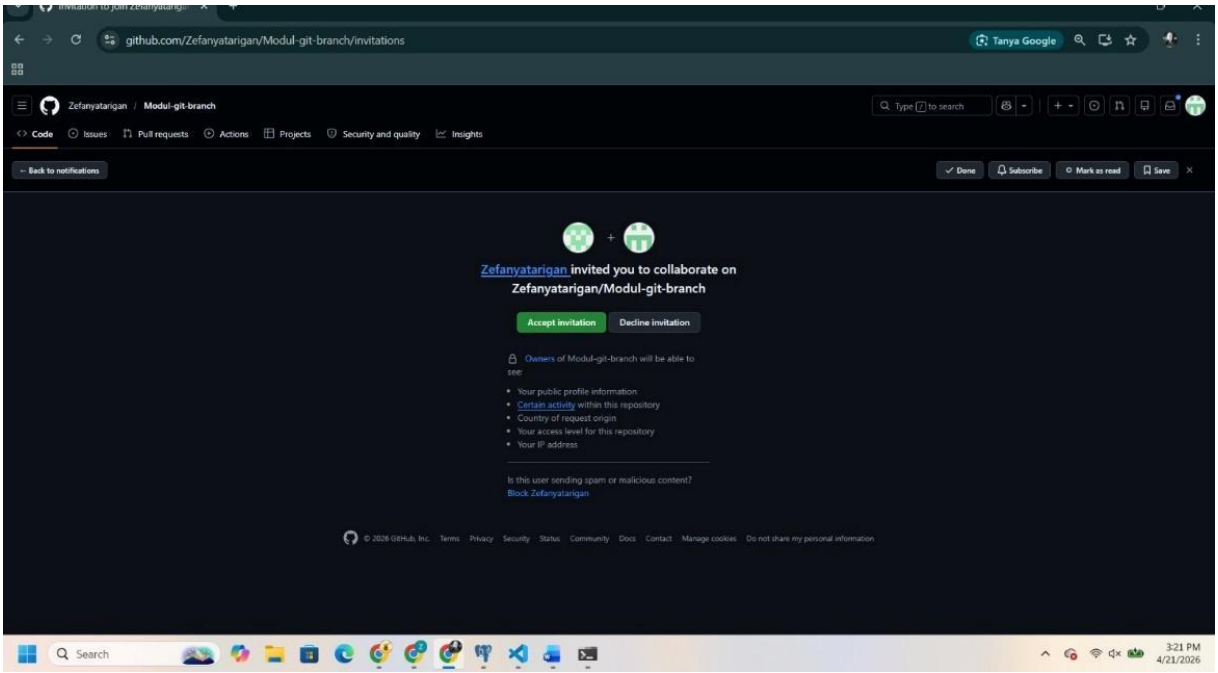
```
http://localhost:3000/api/status
```

Output pengerjaan

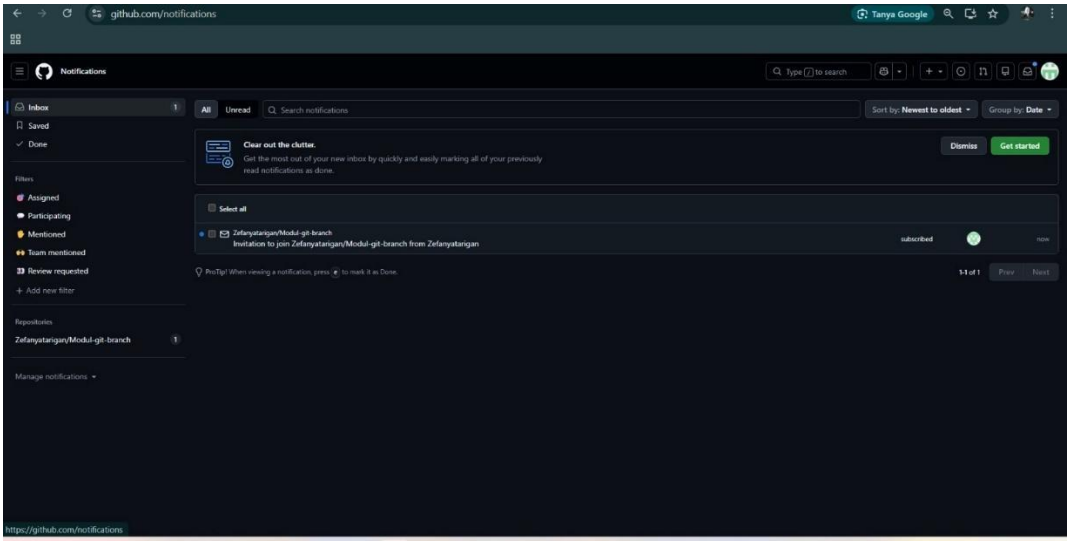



localhost:3000
Data Produk
Nama Produk
Harga
Stok
Simpan
Daftar Produk
Baygon | Harga: 17000 | Stok: 4

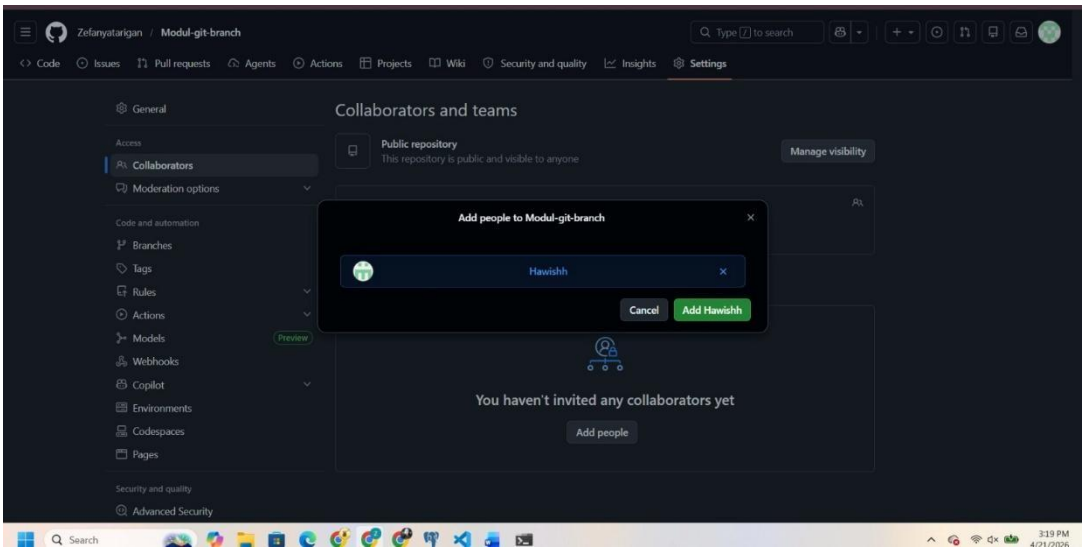

pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
tgres/postgres...
db_tugas2/postgr...
db_tugas2/postgres@PostgreSQL 18*
db_tugas2/postgres@PostgreSQL 18
No limit
Servers
PostgreSQL 18
Databases (2)
db_tugas2
Casts
Catalogs (2)
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (1)
produk
Query
Query History
Scratch Pad
SELECT * FROM produk;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 1
Page No: 1
of 1
id [PK] integer
nama_produk character varying (100)
harga integer
stok integer
1 1 Baygon 17000 4
Total rows: 1
Query complete 00:00:00.761
CRLF
Ln 1, Col 22



Output mengajak collaborators

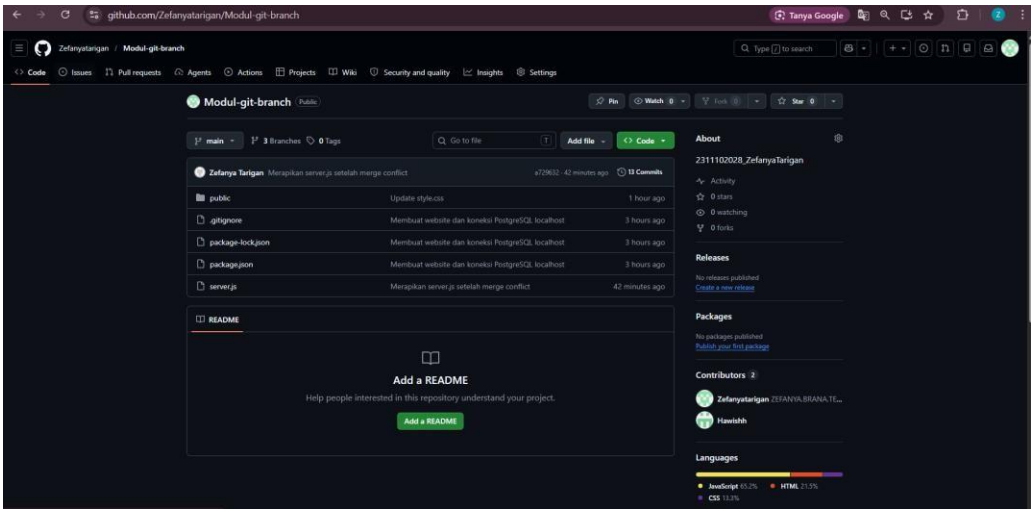


notifikasi nya

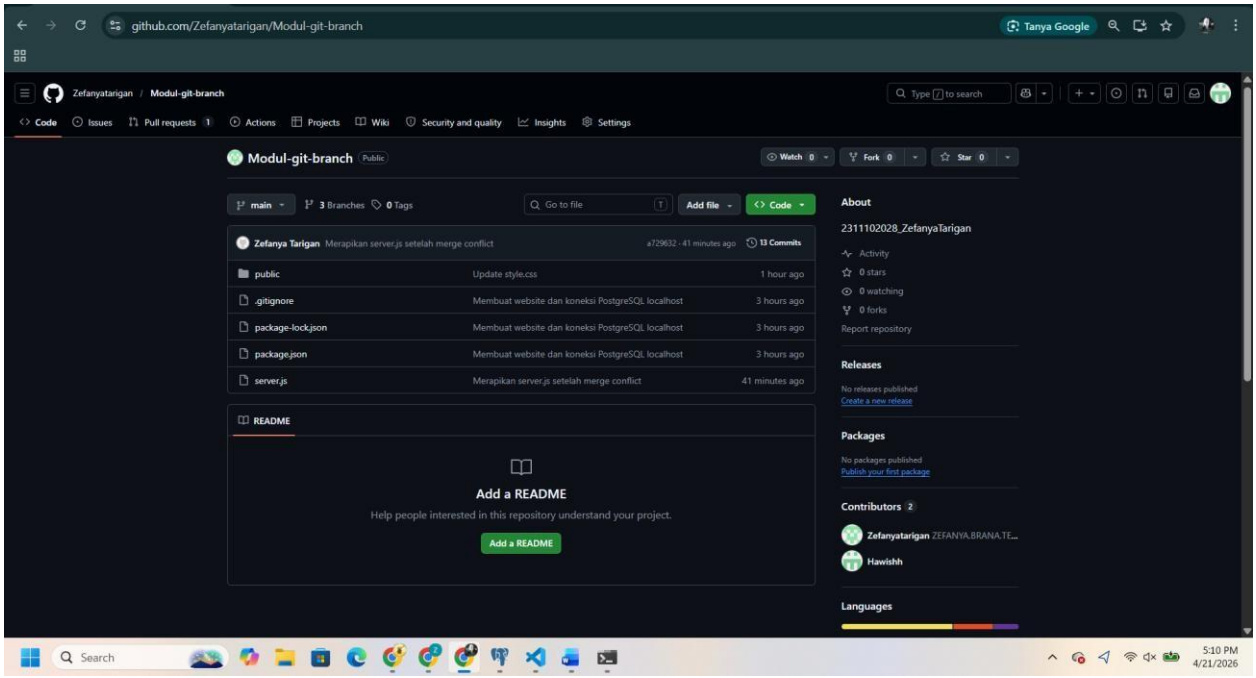


menambahkan

Pada akun ke 1



Pada akun ke 2



sudah menampilkan git yang sudah di push dari akun ke 1